AI-Powered Academic Admission Assistance, Visitor Journey & Queue Management System

Technical Project Explanation & Architecture Documentation

Receptionist Registration • Digital Queue Management • Multi-Stage Admission Workflows • AI Admission Assistant • WhatsApp-Ready Communication

Project: Functional Academic Prototype (FastAPI + React + SQLite + Ollama)

Document: Technical Explanation & Design Documentation

Version: 1.0 | **Date:** September 2026

Status: Implemented, End-to-End Tested (31 automated checks passing)

Table of Contents

1. Project Purpose
2. Core Concept: The Permanent Visitor ID
3. Complete End-to-End Visitor Flow
4. QR Code Access (Planned / Integration-Ready)
5. Real WhatsApp Integration (Meta Cloud API)
6. Local AI with Ollama
7. AI Knowledge Base
8. Unknown Department / Help Desk Routing
9. Receptionist Module
10. Queue Management (8 Statuses)
11. Multi-Stage Workflow Engine
12. Automatic Next-Stage Allocation
13. Staff Dashboard & Actions
14. Admin Dashboard
15. Database Design (14 Tables)
16. Backend Architecture (FastAPI)
17. Frontend Architecture (React + Vite + Tailwind)
18. Authentication & Security
19. Webhook Architecture
20. Notification System
21. Complete Working Example (Visitor V121)
22. Overall System Architecture
23. Detailed Data Flows (A–H)
24. Technology Stack Summary
25. Project Features Summary
26. Testing & Verification
27. Limitations
28. Future Enhancements

> **Accuracy note:** Every section of this document describes the *actual implemented project*. Items that belong to the conceptual "production / college deployment" design but are *not bundled in the current codebase* are explicitly labelled `Planned / Integration-ready`. Implemented capabilities are labelled `Implemented`.

1. Project Purpose

During the admission season, colleges receive many parents and students who visit in person. Most of them do not know:

- whom they should meet for their admission stage,
- which department handles their query,
- where the required counter or room is located,
- how long they will have to wait, and
- what they must do after finishing one admission stage.

This project solves that problem with a single integrated system that combines:

- receptionist-based visitor registration,
- a permanent Visitor ID for every visitor,
- digital queue management (one real queue per admission stage),
- configurable multi-stage admission workflows per department,
- automatic allocation of the next stage when a staff member completes a stage,

- an AI admission assistant backed by an admin-managed college knowledge base,
- WhatsApp-style chat for the parent, with a production-ready WhatsApp API webhook, and
- dedicated Admin, Receptionist and Staff dashboards.

It is important to understand that **this is not a chatbot project**. The core of the system is a **visitor journey + queue + workflow engine**. The AI is one component of that system: it explains, guides and answers, but it never controls the official visitor state.

2. Core Concept: The Permanent Visitor ID

Every visitor receives **one permanent Visitor ID** when they register at the reception. This ID stays with the visitor for the entire visit, no matter how many admission stages or queues they pass through.

```
Visitor ID : V121
Name       : Raj Kumar
Phone      : +91XXXXXXXXXX
Purpose    : Admission
Department : ECE (Electronics & Communication Engineering)

V121 → Counselling → Eligibility Verification → Document Verification
     → Accounts → Admission Confirmation → Registration → COMPLETED
```

Although the visitor is placed into a *different queue with a different token number at each stage* (for example `ECE-COUNS-124`, then `ECE-ELIG-…`, then `ECE-DOC-…`), the permanent Visitor ID never changes.

Why this design is useful:

- **Traceability:** staff, admin and the AI can always refer to one stable identifier across the entire journey and audit history.
- **Simpler communication:** WhatsApp/chat notifications always refer to the same visitor, e.g. "V121: your turn is now".
- **Accurate reporting:** statistics such as "visitors per department", "stage progression time" and "completed admissions" are computed per visitor, not per token.
- **Clean audits:** the `visitor_stage_history` table records every stage a single Visitor ID passed through, with timestamps and the staff member who acted.

3. Complete End-to-End Visitor Flow

The system is understood best by following the complete conceptual flow, from a visitor arriving at the college to final completion.

```
PARENT / STUDENT ARRIVES
        |
        ▼
QR CODE (access mechanism → opens chat/WhatsApp entry point)
        |
        ▼
WHATSAPP (real WhatsApp via Cloud API, or built-in chat UI for local demo)
        |
        ▼
AI ADMISSION ASSISTANT (answers questions, guides, reports queue status)
        |
        ▼
RECEPTIONIST REGISTRATION (name, mobile, purpose, department, remarks)
        |
        ▼
VISITOR ID (e.g. V121) created
        |
        ▼
PURPOSE / DEPARTMENT IDENTIFICATION (known dept → its workflow; unsure → Help Desk)
        |
        ▼
INITIAL QUEUE (visitor placed in first stage queue, e.g. ECE Counselling)
        |
        ▼
STAGE PROCESSING (staff member assigned to that stage sees the visitor)
        |
        ▼
STAGE COMPLETION (staff clicks COMPLETE)
        |
        ▼
AUTOMATIC NEXT STAGE (workflow engine finds the next stage)
        |
```

```
        ▼
NEXT QUEUE (visitor automatically added to the next stage's queue)
        │
        ▼
WHATSAPP NOTIFICATION ("Verification done. Next step: Document Verification…")
        │
        ▼
NEXT STAFF / ROOM (visitor walks to the newly assigned counter/room)
        │
        ▼
FINAL COMPLETION (last stage done → visitor status = COMPLETED)
```

1. **Visitor arrives.** The parent/student walks into the college.
2. **QR code (Planned/Integration-ready).** A printed QR is the convenient entry point that opens the WhatsApp chat / chat UI so the AI assistant becomes available immediately. It is simply an access mechanism, not the AI itself (see Section 4).
3. **AI conversation** answers early questions (documents, fees, hostel, timings, location).
4. **Reception registration** connects the physical visitor to the digital journey and creates the Visitor ID.
5. **Queue assignment.** The workflow engine selects the correct first-stage queue.
6. **Staff processing** and **automatic next-stage allocation** repeat across the workflow.
7. **Notifications** at every transition keep the parent informed of their turn, the room, the staff member and the next step.

4. QR Code Access Planned / Integration-ready

The QR code is the simplest possible entry point for parents and students. It is **only an access mechanism** — it does not perform any AI processing itself.

```
QR SCAN
  │
  ▼
Opens the configured WhatsApp communication entry point
  │
  ▼
Parent sends a message (or taps a pre-filled quick message)
  │
  ▼
AI admission assistant becomes available
```

- The QR encodes a link (for example a WhatsApp `wa.me` deep-link, or a URL of the built-in chat page / a hosted web-widget).
- Scanning it lands the parent directly in the conversation with the admission assistant, optionally pre-filled with a starter message such as "Hi, I came for admission."
- It removes all friction for a parent who would otherwise have to type a number or look for the right phone number.
- Because it is a printed static object that points to a configured link, it works with both the future real WhatsApp integration and the currently bundled chat UI.

> **Accuracy:** the QR code is *not generated or scanned in the current codebase*. It is part of the deployment design; the backend/frontend already expose the chat entry point (`/chat?phone=…`) that such a QR would link to.

5. Real WhatsApp Integration (Meta Cloud API)

The project is designed around **real WhatsApp communication** using Facebook/Meta's official **WhatsApp Business Platform / Cloud API**. It does not use any unofficial automation method.

```
Parent WhatsApp
      │
      ▼
Meta WhatsApp Cloud API
      │
      ▼
Webhook endpoint  (GET verify + POST receive)
      │
      ▼
FastAPI Backend (AI + Queue + Workflow + Database)
      │
      ▼
FastAPI → WhatsApp Cloud API (outbound message)
      │
```

```
      ▼
Parent WhatsApp
```

How incoming messages work

- When a parent sends a WhatsApp message, Meta forwards it to the configured `/api/whatsapp/webhook` endpoint.
- The backend extracts the sender's phone number and the message text (text, button and interactive message types are handled).
- The visitor is looked up by phone number. If found, the AI service generates an answer using the visitor's live context and the knowledge base.
- The message pair (USER + AI) is stored in the `chat_messages` table so the conversation history is retained.
- The reply is sent back through the WhatsApp Cloud API using an access token and the phone-number ID from the environment.

Credentials stay in the environment

Nothing is hard-coded. The following are read from `.env` / environment variables (see Section 18):

```
WHATSAPP_ACCESS_TOKEN     - Meta API access token
WHATSAPP_PHONE_NUMBER_ID  - the registered WhatsApp business phone number ID
WHATSAPP_VERIFY_TOKEN     - token used to verify the webhook during setup
WHATSAPP_API_BASE         - Cloud API base URL (default: graph.facebook.com v19.0)
```

When these are not configured, the back-end keeps working through the **built-in chat UI**, which uses the exact same backend processing path (`/api/chat/send` mirrors the webhook logic). This is what makes the project fully demonstrable locally.

6. Local AI with Ollama

The AI runs **locally** — the project does not depend on any paid cloud AI API. The AI layer is built around a clean service abstraction (`AIService`) with two concrete providers:

- `RulesBasedAIService` — an offline, deterministic assistant that answers from the configured college knowledge base and reports live queue/workflow state.
- `OllamaAIService` — delegates understanding to a local large language model served by **Ollama**, while *injecting the same knowledge base and live visitor context into the prompt*. If Ollama is unreachable, it automatically falls back to the rules-based service.

```
User message
      |
      ▼
FastAPI (/api/chat/send or WhatsApp webhook)
      |
      ▼
AI Service (AIService interface → factory picks provider)
      |
      ▼
Ollama (http://localhost:11434/api/chat)
      |
      ▼
Local LLM (model from OLLAMA_MODEL)
      |
      ▼
Generated response
      |
      ▼
FastAPI → WhatsApp / Chat UI
```

What Ollama does

Ollama is a local LLM runner. It exposes a REST interface (`/api/chat`) through which the backend sends a message plus context and receives a generated reply. The selected model is configured by the `OLLAMA_MODEL` environment variable; the bundled default value is `llama3.2`. Because the model is env-configurable and the service class is swappable, the underlying model can be changed without touching any route code.

Why a local AI approach fits this project

- **No fees / no API keys:** the assistant runs fully offline out of the box (rules mode) or on a local model.
- **Privacy:** visitor data stays inside the college network — no messages are sent to third-party servers.

- **Deterministic fallback:** the rules provider guarantees an answer even without any model installed.
- **Modularity:** the same interface can later add other providers without changing the rest of the system.

> **Accuracy:** the exact default model name in the code is `llama3.2` (config default, not hard- coded in route logic). The *local LLM is integration-ready*; the default running behaviour in the demo is the rules-based provider.

7. AI Knowledge Base

The AI does not imagine college information. All authoritative facts live in a **database-driven knowledge base** that an administrator manages from the Admin dashboard. It can contain:

- college details and contact information,
- courses and departments,
- admission information, eligibility criteria and fees,
- required documents,
- hostel and transport information,
- campus timings, and
- room / counter / building locations and FAQs.

Two tables store this content: `faqs` (question → answer, optionally tied to a department) and `college_information` (key/category/title/content, e.g. "help_desk", "Hostel", "Transport", "Location").

The AI answers questions such as:

- "What documents are required for ECE admission?" — answered from the departmental FAQ / knowledge base.
- "Where is the admission office?" — answered from location/college information.
- "What is the fee?" — from the fees entry.
- "Is hostel available?" — from hostel information.
- "How many people are before me?" — from *live queue data*, not the knowledge base.

Read-only AI — clear responsibilities

```
AI                → understands the question, communicates, guides
Backend / database → provides authoritative system information (KB, status)
Queue / workflow   → controls the actual visitor state (staff & reception only)
```

The AI service is **strictly read-only**. It can report a queue position, a current stage or a next step, but it can never edit a queue entry or move a visitor. All official state changes go through the queue/workflow engine triggered by authorized staff or receptionist actions. This rule is enforced at the architectural level: the AI service modules never import or call the mutating functions of the queue/workflow engine.

8. Unknown Department / Help Desk Routing

A very common, and intentionally supported, situation is the visitor who does not know which department they need. When a receptionist registers such a visitor (or the AI detects the confusion in chat), the system does **not** guess a department.

```
Parent: "I don't know which department I need."
        │
        ▼
AI: No problem — our counselling/help desk team can guide you based on
    your interests and marks. We will NOT assign you to any department
    automatically.
        │
        ▼
Reception registers with Department = "Needs Counselling / Help Desk"
        │
        ▼
Workflow engine assigns the dedicated "Counselling / Help Desk" workflow
        │
        ▼
Visitor is queued behind the counsellor/help-desk officer
```

- A dedicated **"Counselling / Help Desk"** workflow (with its own one-stage queue) is defined in the database, just like the department workflows.
- In chat, keywords such as "which department", "not sure", "can't decide", "guide me" and "I don't know which" trigger the help-desk guidance answer.

- A counsellor then interviews the visitor in person and helps them choose the right programme.

Why this improves the experience: randomly pushing an unsure visitor into a department would create wrong queues, wrong staff work and a confusing journey. Help-desk routing guarantees accuracy, and it is handled entirely by the same queue/workflow machinery — no special-casing in the AI.

9. Receptionist Module

The receptionist is the human bridge between the physical visitor and the digital journey. The reception dashboard provides:

- register a new visitor,
- search visitors by name/code/phone,
- view all active visitors,
- view all queues across the college,
- see each visitor's current workflow stage and status, and
- reassign a visitor to a different stage when needed.

```
NEW VISITOR REGISTRATION
----------------------------------------
 Name       : Raj Kumar
 Mobile     : 9876543210
 Purpose    : Admission
 Department : ECE               (or "Needs Counselling / Help Desk")
 Remarks    : Walk-in parent
----------------------------------------
[REGISTER VISITOR]   →   generates Visitor ID V121
```

Behind the `REGISTER` button the backend:

1. validates the data and creates the `Visitor` record with a unique sequential Visitor ID (`V121`, `V122`, ...),
2. resolves the appropriate workflow (department workflow, or Help Desk workflow when unknown),
3. loads the first active stage of that workflow,
4. places the visitor into that stage's queue with a fresh token (e.g. `ECE-COUNS-124`),
5. creates a `VisitorStageHistory` entry, and
6. emits a welcome notification/message containing the Visitor ID, current stage, queue token and instructions.

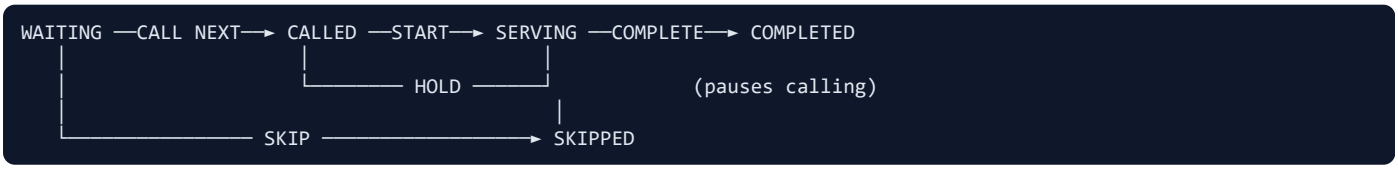
This is exactly how the physical visitor becomes an entity the AI, the queues and the dashboards all understand.

10. Queue Management (8 Statuses)

Each active workflow stage has a **real queue** backed by database rows — not a static list. A stage in the `workflow_stages` table is linked to a `Queue` record, and every visitor currently in that stage holds a `QueueEntry` with a token.

Status	Meaning	Who sets it
WAITING	Visitor is in the queue, waiting for their turn.	Queue engine (register / next stage)
CALLED	Staff pressed CALL NEXT; the visitor has been summoned.	Staff action
SERVING	Staff began serving the visitor (start-serving).	Staff action
COMPLETED	Current stage finished; visitor may be auto-moved to next queue.	Staff COMPLETE
HOLD	Processing paused for this visitor; they are not called next.	Staff action
SKIPPED	Visitor moved out of this queue (e.g. came back later / rescheduled).	Staff action
CANCELLED	Visitor record cancelled (no-show / left college).	Reception / staff

Transitions on a single queue entry:



Queue position (people ahead): computed live by the queue engine by counting `WAITING` entries that arrive earlier in the same queue. The AI and the chat UI both report this real number — a visitor who asks "how many people are before me?" gets the actual current figure.

Calling the next visitor: `CALL NEXT` takes the earliest `WAITING` entry, moves it to `CALLED`, and triggers a "Your turn is now. Please proceed to Room ..." notification.

Every mutation is performed by the backend queue engine invoked only through authorized staff/reception/admin endpoints. The AI has no path to change any of these statuses.

11. Multi-Stage Workflow Engine

The system does not assume every department follows the same process. A **workflow** is a configurable chain of **stages**, and each stage knows its department, staff, room/counter, queue, order and active/inactive state.

Workflow	Defined stages (in order)
ECE Admission	Counselling → Eligibility Verification → Document Verification → Accounts → Admission Confirmation → Registration
CSE Admission	Configurable, separate chain (same engine, different stages/order)
EEE / Mechanical / MBA	Configurable, department-specific chains
Counselling / Help Desk	Single help-desk stage for visitors unsure of their department

The core engine functions are:

- `find_workflow_for_visitor` — picks the right workflow from department / help-desk flag,
- `first_stage` — the first active stage of a workflow,
- `next_stage` / `previous_stage` — neighbour lookup along the stage order,
- `is_visitor_complete` — true when no active next stage exists.

Administrators create and edit workflows, stages, staff and rooms from the Admin dashboard — so the admission process itself is stored as *configuration data*, never as hard-coded logic.

12. Automatic Next-Stage Allocation

This is one of the central technical features. When a staff member completes a visitor's current stage, the backend chains six actions automatically:

```
COMPLETE pressed for V121 at "Counselling"
        │
        ▼
1. Mark current queue entry COMPLETED
        │
        ▼
2. Workflow engine finds next active stage → "Eligibility Verification"
        │
        ▼
3. Visitor is added to the next stage's queue (new token)
        │
        ▼
4. Visitor.current_stage updated to the next stage
        │
        ▼
5. Stage history record written (who, when, from, to)
        │
        ▼
6. Notification: "Verification done. Next step: Eligibility Verification.
                  You are in the queue. Room: Eligibility Counter."
```

```
Example (ECE workflow):

Counselling           Document Verification
   V120 → SERVING          V120 → SERVING
   V121 → WAITING          V121 → WAITING(V121 ECE-ELIG-…)

   [COMPLETE]              [COMPLETE]
   V120 → COMPLETED        V120 → COMPLETED
   V121 → CALLED           V121 → CALLED
   notification to V121    notification to V121
   "Your turn, Room 102"   "Next stage reached…"
```

The same chain repeats until the last stage. At the final stage there is no next stage, so the visitor's overall status becomes `COMPLETED` and a congratulations notification is sent.

13. Staff Dashboard & Actions

Each staff account is bound to staff records that map to specific workflow stages. The staff dashboard therefore shows only the queues the logged-in staff member is *assigned* to. For every queue it displays:

- stage name and workflow, room/counter, running queue count,
- the currently serving visitor, and the waiting list with tokens, names and positions,
- visitor details (name, phone, purpose, department, remarks, history).

The action bar: `CALL NEXT` `HOLD` `SKIP` `REASSIGN` `VIEW` `COMPLETE`. Every button performs a real backend action:

Action	Backend effect
CALL NEXT	Earliest WAITING entry → CALLED; notification "Your turn is now…" sent to the visitor.
START	Entry CALLED → SERVING with a served-at timestamp.
COMPLETE	Runs the auto next-stage chain (Section 12) — moves visitor to the next queue.
HOLD	Entry → HOLD (with reason); skipped by the next CALL so others are served first.
SKIP	Entry → SKIPPED (with reason); visitor leaves this queue run.
REASSIGN	Visitor moved to a chosen workflow stage; old entry cancelled, new entry created in the target queue.
VIEW	Read-only modal with the visitor's details, stage history and chat/notification trail.

All actions write audit info (which user acted, timestamps, reasons) so the journey is fully reconstructable.

14. Admin Dashboard

The Admin dashboard makes the whole system **configurable instead of hard-coded**. The administrator can manage:

- **Departments** — add/edit departments,
- **Rooms & counters** — buildings, floors, room codes,
- **Staff** — create staff profiles and bind them to users and stages,
- **Workflows** — create department workflows,
- **Workflow stages** — define stage names, order, room, assigned staff, queue and active flag,
- **FAQs** — knowledge-base Q&A entries,
- **College information** — hostel/transport/fees/timings/location entries,
- **Queues & visitors** — monitoring and manual reassignment,
- **Statistics** — total visitors today, waiting, serving, completed, visitors by department, average wait, pending stage visitors.

A workflow's behaviour can therefore change with a few clicks — e.g. adding a "Photograph & Signature Upload" stage to ECE, or changing the room of the Accounts counter — without any code change. The FAQ and college-info editors directly feed the AI's knowledge base.

15. Database Design (14 Tables)

Storage uses **SQLite via SQLAlchemy ORM** — simple, reliable and perfectly suited to an academic prototype, while the ORM allows a later move to PostgreSQL with minimal change. The schema contains exactly 14 tables:

Table	Stores	Key relationships
`departments`	CSE, ECE, EEE, Mechanical, MBA, …	→ workflows, visitors, staff
`rooms`	Rooms/counters, building, floor, code	→ workflow_stages
`users`	Login accounts: admin, reception, counsellor, registrar, …	→ staff
`staff`	Staff profile, department, designation	→ users, workflow_stages
`workflows`	Named admission workflows per department or help desk	→ departments, workflow_stages
`workflow_stages`	Stage name, order, room, assigned staff, active flag	→ workflows, rooms, staff, queues

`queues`	One real queue per active stage	→ workflow_stages, queue_entries
`queue_entries`	Visitor tokens & status inside a queue	→ queues, visitors
`visitors`	The permanent visitor record (V121 …), phone, purpose, department, status	→ queues/entries, history, chats
`visitor_stage_history`	Audit trail of every stage change per visitor	→ visitors, workflow_stages
`faqs`	Knowledge base Q&A, optionally per-department	→ departments (optional)
`college_information`	College/hostel/fees/timings/location/help-desk entries	—
`notifications`	Per-visitor notification log	→ visitors
`chat_messages`	Conversation history (USER/AI/SYSTEM)	→ visitors

Example relationship chain: `Visitor → current_queue_entry → QueueEntry → Queue → WorkflowStage → Workflow → Department`, plus `VisitorStageHistory` for the audit trail and `Notifications / ChatMessages` for communication. These real relationships are what make queries like "full journey of V121", "people ahead in this queue" and "pending visitors per stage" fast and reliable.

16. Backend Architecture (FastAPI)

FastAPI is the central application layer. It exposes REST APIs, enforces authentication, orchestrates the queue/workflow engines, talks to SQLite through SQLAlchemy, generates AI answers and hosts the WhatsApp webhook — all under one process.

Router modules (API endpoints)

Router	Responsibilities (representative endpoints)
`auth_routes`	`POST /api/auth/login`, `GET /api/auth/me`
`reception_routes`	registration, departments, visitors + search + active, all queues, stage-map, cancel, reassignment
`staff_routes`	my queues, call-next, start-serving, complete, hold, skip, reassign, details
`admin_routes`	stats; CRUD for departments, rooms, staff, workflows, stages, FAQs, college-info; reassign
`chat_routes`	chat session, send message, visitor lookup, knowledge-base info
`whatsapp_routes`	GET webhook verify, POST webhook receive, status

Service layer

Service	Role
`queue_engine`	Get-or-create queues, enqueue, positions, next waiting, stage summary
`workflow_engine`	Workflow lookup, first/next/previous stage, completion checks
`visitor_flow`	High-level operations: register, call-next, complete, hold, skip, reassign, next visitor codes
`ai_service`	Rules-based + Ollama provider, visitor context builder, factory
`notification_service`	Create notification rows + system chat messages
`whatsapp_service`	Cloud API outbound sender (enabled only when credentials exist)
`helpers`	Visitor/queue serialization for API responses

Startup behaviour: `main.py` runs `init_db()` (creates the SQLite file/tables) and `seed_all()` which loads the realistic demo data, so the project is demonstrable immediately after launch.

17. Frontend Architecture (React + Vite + Tailwind)

The frontend is a single-page application built with **React 18, Vite and Tailwind CSS**. Vite provides the dev server (port 5173) and the production build; a dev proxy forwards `/api` calls to the FastAPI backend on port 8000.

Structure

- `src/lib/api.js` — axios instance with automatic JWT header injection,
- `src/lib/auth.jsx` — AuthContext, login state and role guards,

- `src/components/Layout.jsx` — sidebar + topbar shell,
- `src/components/ui.jsx` — reusable Badge, StatCard, Modal, Btn, Spinner, inputs,
- `src/pages/Login.jsx` — role-aware login,
- `src/pages/Receptionist.jsx` — registration, active visitors, all queues,
- `src/pages/Staff.jsx` — live queue console with all six actions,
- `src/pages/Admin.jsx` — stats + CRUD tabs,
- `src/pages/Chat.jsx` — WhatsApp-style parent chat with a live tracker (Visitor ID, stage, position, next step).

The UI uses tables, status badges, stat cards, search, filters and modal forms — all styled with Tailwind. The parent chat is a simple, mobile-friendly mobile-style interface. The three role interfaces verify permissions between frontend and backend: the shell shows different menus per role, and the backend independently enforces each role's access.

18. Authentication & Security

Authentication uses **JWT (JSON Web Tokens)** with **PBKDF2 password hashing** (PyJWT). Login returns a signed token the frontend stores and sends as `Authorization: Bearer <token>`; every restricted endpoint validates it.

Role	Can access
ADMIN	Full access incl. departments, rooms, staff, workflows, stages, FAQs, college info, stats, visitor records.
RECEPTIONIST	Visitor registration, search, active visitors, all queues, stage map, reassignment, cancellation.
STAFF	Only the queues/stages assigned to them, and processing actions (call, start, complete, hold, skip, reassign).

Backend guards (`get_current_user`, `require_roles`, `require_staff`) are applied per route. The frontend additionally hides pages/buttons per role, but the backend is the real enforcement point.

Environment & secrets

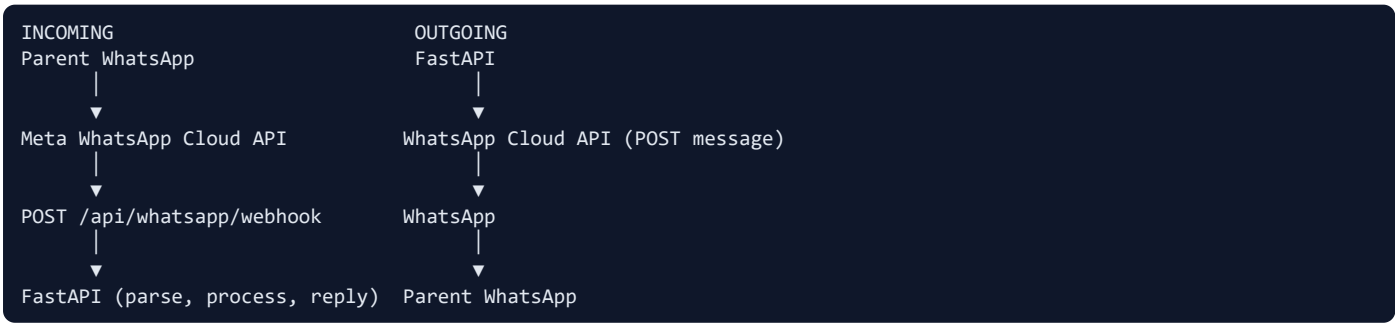
The project ships a `.env.example` listing every variable; the real `.env` is created locally and must never be committed. Key variables (no real values shown):

Variable	Purpose
`HOST` / `PORT`	Server binding
`SECRET_KEY`	JWT signing secret (must be changed from default)
`DATABASE_URL`	SQLite (or other) database URL
`AI_PROVIDER`	`rules` (default) or `ollama`
`OLLAMA_BASE_URL`, `OLLAMA_MODEL`	Local Ollama endpoint and model
`WHATSAPP_ACCESS_TOKEN`, `WHATSAPP_PHONE_NUMBER_ID`, `WHATSAPP_VERIFY_TOKEN`	WhatsApp Cloud API credentials — never hard-coded

Demo login accounts (intentionally public for the academic demo): `admin` / `reception` / staff users `priya`, `arun`, `kavita`, `suresh`, `anita`, `vikram`, `divya` (demo password `…123` pattern per the README). Incremental demo changes to these demo credentials and the JWT secret should be applied in the `.env`/seeder before any real use.

19. Webhook Architecture

A webhook is a URL your backend exposes so that a third party (Meta) can push data to it when an event occurs — here, an incoming WhatsApp message.



- **Webhook verification (GET):** during setup, Meta sends a `hub.mode`, `hub.verify_token` and `hub.challenge`. The endpoint accepts the request only when the verify token matches `WHATSAPP_VERIFY_TOKEN`; otherwise it returns 403. This confirms to Meta that the webhook is really controlled by us.
- **Inbound (POST):** message payloads are parsed (text / button / interactive types), the visitor is resolved by phone, the AI answers, the conversation is stored, and the reply is sent back.
- **Why HTTPS/public access is required:** Facebook only delivers webhooks to publicly reachable HTTPS URLs. During the local academic demo the equivalent behaviour runs through the built-in chat UI; going live needs a public HTTPS endpoint and configured Meta app credentials — both are covered by this implementation, which is why it is marked Integration-ready.

20. Notification System

Every meaningful backend event pushes a notification to the visitor's conversation and notification log:

- **Registration confirmation** — Visitor ID, stage, queue token and instructions,
- **Queue position answers** — generated live by the AI on request,
- **Turn notification** — "Your turn is now! ECE-COUNS-124. Please proceed to room / meet staff",
- **Next-stage notification** — "... You have been added to the Document Verification queue. Please wait for your turn",
- **Location/staff instruction** — room/counter and assigned officer for each stage,
- **Final completion** — "All your admission stages are complete. Congratulations!"

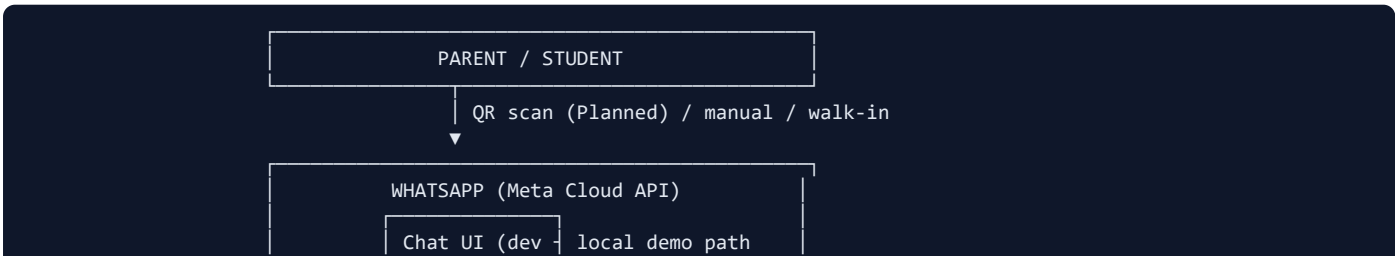
Technically, `push_notification()` does two things at once: (1) writes a `Notification` row and a system `ChatMessage` (always available in the chat UI), and (2) if WhatsApp credentials are configured, sends the same text through the WhatsApp Cloud API. Because notifications originate from the workflow/queue engine itself, they always reflect the true official state — never an AI guess.

21. Complete Working Example — Visitor V121 (Raj Kumar, ECE)

```
Reception:   Raj Kumar · 9876543210 · Purpose: Admission · Dept: ECE
     │   [REGISTER]
     ▼
V121 created → ECE Admission workflow → stage 1 "Counselling"
     │
     ▼
Queue: ECE-COUNS-124  (WAITING)
     │
     ▼
Chat:   Parent : "What documents are required for ECE admission?"
        AI     : "Mark sheets & certificates, transfer certificate, …"
        Parent : "How many people are before me?"
        AI     : "V121: number ECE-COUNS-124. 0 people ahead — you are next."
     │
     ▼
Staff (Priya @ Counselling): CALL NEXT
     │  → V121 CALLED → "Your turn is now! ECE-COUNS-124. Proceed to
     ▼     Counselling Room (R101). Meet: Mrs. Priya Sharma."
COMPLETE → auto next stage: Eligibility Verification → new queue token
     │  → "Verification done. Next step: Eligibility Verification.
     ▼     You have been added to the queue (Room: Eligibility Counter)."
…chain continues: Document Verification → Accounts → Admission Confirmation
     ▼
Registration (last stage) → COMPLETED
     │  → "All your admission stages are complete. Congratulations!"
```

This exact journey also exists as an automated test (Section 26). A second visitor (Sundar Kumar) registers with "not sure" and is correctly routed to the Counselling / Help Desk workflow instead of any department.

22. Overall System Architecture





Layer explanations. The *parent/student layer* is the end user. The *access layer* is WhatsApp (production) with the QR as an entry point, or the built-in chat UI for the local demo. The *integration layer* is the Meta Cloud API + webhook. The *core layer* is FastAPI with its services (AI, queue, workflow, visitor flow, notifications). The *data layer* is SQLite with 14 tables. The *management layer* is the role-based React frontend. Notifications travel back through the WhatsApp service to the parent.

23. Detailed Data Flows (A–H)

A. Visitor registration

```
Reception UI → POST /api/reception/visitors → validate → create Visitor (V121)
  → resolve workflow → first stage → create QueueEntry (ECE-COUNS-124)
  → create StageHistory → push welcome notification → JSON response to UI
```

B. AI question answering

```
Parent message → /api/chat/send → load visitor by phone → AIService.answer
  → rules: KB match (FAQ / college_info) or Ollama prompt with KB + live context
  → store ChatMessages (USER, AI) → return answer
```

C. Queue position query

```
"How many people are before me?" → AI builder reads visitor.current_queue_entry
  → queue_engine.position() counts earlier WAITING entries → answer text
```

D. Staff CALL NEXT

```
Staff UI → POST /api/staff/queues/{id}/call-next → earliest WAITING → CALLED
  → push "Your turn" notification → updated entry returned to UI
```

E. Staff COMPLETE

```
POST /api/staff/entries/{id}/complete → mark entry COMPLETED
  → record history → return updated entry
```

F. Automatic next-stage allocation

```
complete_stage() → workflow_engine.next_stage(current)
  → if none: visitor.status = COMPLETED
```

```
  → else: enqueue into next stage queue → visitor.current_stage updated
  → history row → next-stage notification
```

G. WhatsApp notification

```
push_notification() → notify() writes Notification + ChatMessage
  → if WHATSAPP_ACCESS_TOKEN set: WhatsAppService.send_text(phone, text)
  → Cloud API → parent phone
```

H. Help-desk routing

```
department_id = null / needs_help_desk → workflow "Counselling / Help Desk"
  → single-stage queue → counsellor (staff) handles → AI guides, never auto-assigns
```

24. Technology Stack Summary

Layer	Technology	Purpose in the project
Frontend	React 18 (react-dom, react-router-dom), Vite 5	SPA UI, routing, dev/build tooling
Styling	Tailwind CSS 3, postcss, autoprefixer	Responsive enterprise/academic UI
Frontend libs	axios, lucide-react	HTTP client, icons
Backend	Python 3.10+, FastAPI (uvicorn)	REST API, webhook, central application layer
Database	SQLite + SQLAlchemy 2 (ORM), pydantic v2	14-table relational schema + validation
Auth	PyJWT (HS256), PBKDF2 hashing (standard lib)	JWT tokens, password hashing, role guards
AI	Rules-based service (default) + Ollama REST (optional local LLM)	Admission assistant; read-only re: queues
WhatsApp	Meta WhatsApp Cloud API (webhook + REST outbound)	Real parent messaging (integration-ready)
HTTP/other	httpx, python-dotenv, python-multipart	Ollama/WhatsApp calls, env config, form parsing
Testing	FastAPI TestClient (pytest-style script)	31-check end-to-end automated test

25. Project Features Summary

Feature	Coverage
QR access	Planned / Integration-ready — entry point to chat/WhatsApp
Real WhatsApp (Cloud API)	Implemented service + webhook; live delivery needs credentials (Integration-ready)
Local AI (Ollama)	Implemented service with rules fallback; local LLM optional
College knowledge base	Implemented (faqs + college_information, admin-managed)
Permanent Visitor ID	Implemented (V121 ...)
Reception registration	Implemented
Queue management	Implemented ; real positions, people ahead
8 queue statuses	Implemented (WAITING, CALLED, SERVING, COMPLETED, HOLD, SKIPPED, CANCELLED)
Multi-stage workflows	Implemented (configurable per department)
Department-specific workflows	Implemented (CSE/ECE/EEE/MECH/MBA)
Help-desk routing	Implemented
Admin / Reception / Staff dashboards	Implemented
Automatic next-stage allocation	Implemented
Notifications	Implemented (DB + chat + optional WhatsApp)
JWT auth + hashing + RBAC	Implemented
Parent chat UI	Implemented (WhatsApp-style, local demo)
Demo data / instant demo	Implemented (auto-seed on startup)

26. Testing & Verification

The project has been exercised end-to-end. Verified results:

Verification	Result
Backend boots (uvicorn)	PASS — server starts, startup creates DB and seeds
Database seeding	PASS — departments, rooms, users, staff, workflows, FAQs, college info, visitors
Login (admin / reception / staff)	PASS — valid tokens issued, bad passwords rejected
Frontend production build	PASS — `vite build` succeeds; dev server serves on :5173
End-to-end automated flow test	PASS — **31 checks** in `backend/tests/test_flow.py`
HOLD / SKIP / REASSIGN actions	PASS — statuses and stage-move verified by the test suite
Full journey V121 (six ECE stages)	PASS — counselling → … → registration → COMPLETED, with notifications
Help-desk routing (unknown department)	PASS — routed to Counselling / Help Desk, AI guidance verified
AI knowledge-base answers + live queue answers	PASS — documents, position, status verified

> **Live WhatsApp delivery** is the one integration that requires real Meta/WhatsApp credentials and a public HTTPS deployment; it has *not* been claimed as verified without credentials. The webhook, verification endpoint, outbound sender and env-based configuration are all implemented and unit-exercisable, but actual message delivery is expected only after proper Meta Business Platform configuration.

27. Limitations

- **Local AI quality** depends on the selected Ollama model and the hardware it runs on; the bundled rules provider is always available as a baseline.
- **Real WhatsApp deployment** requires a configured Meta Business Platform account, valid access token, phone-number ID and a public HTTPS webhook.
- **Knowledge base accuracy** depends on the college keeping FAQs and college information up to date in the Admin dashboard.
- **Queue estimates** depend on actual staff processing times; position is accurate per current queue data, wait time is indicative.
- **Connectivity:** WhatsApp delivery requires internet; the chat-UI fallback is fully offline/local.
- **Production hardening:** a real deployment needs HTTPS, secrets rotated (JWT secret, tokens), backups and higher-grade hosting; SQLite suits the prototype scale.

28. Future Enhancements

The following improvements are realistic next steps and are *not* part of the current implementation:

- multilingual AI and multilingual WhatsApp support,
- voice-based assistance (voice messages / speech-to-text),
- admission analytics dashboards and trend reports,
- appointment scheduling for counselling and document submission,
- live occupancy / crowd monitoring per room,
- additional college integrations and cloud deployment,
- a mobile staff application for on-the-go call/complete actions,
- advanced RAG / document retrieval over admission brochures and circulars,
- admission document upload by the parent through chat, and
- automated appointment reminders via WhatsApp.

End of document — AI-Powered Academic Admission Assistance, Visitor Journey & Queue Management System. Prepared from the actual project source code: FastAPI backend (6 router modules, 7 services, 14 tables, auto-seeding), React 18 / Vite 5 / Tailwind 3 frontend (4 role dashboards + chat UI), and a 31-check end-to-end test suite.